



Startup Relámpago: Veterinaria Patitas

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación 3

COMISIÓN: "A"

PROFESOR: Sicardi Dante

ESTUDIANTE: Roa Santino

FECHA: 30-09-2026



INTRODUCCIÓN

El presente trabajo práctico desarrolla el proyecto Veterinaria Patitas, un sitio web para una veterinaria de la ciudad de Posadas que, además de sus servicios profesionales, ofrece una tienda online de productos para mascotas. El trabajo integra dos etapas: por un lado, el análisis y diseño del frontend de la tienda con HTML y CSS; por otro, el módulo de Contacto y Consultas, enriquecido con JavaScript.

En el desarrollo se presentan el requerimiento funcional resuelto, el mapa del sitio, el diseño de cada pantalla, la forma en que se aplicaron los requisitos de JavaScript (captura de eventos con `addEventListener` y asincronía con `fetch` y `async/await`) y la fundamentación del modelo EORM para la relación entre el cliente y el servicio consultado.



DESARROLLO

1. Descripción del proyecto

Veterinaria Patitas es una veterinaria de Posadas, Misiones. El sitio busca dos objetivos: mostrar al público el catálogo de productos (alimentos, sanidad, higiene y accesorios) para que pueda comprarlos desde su casa, y ofrecer un canal simple para enviar consultas sobre los servicios que brinda el local, como consulta general, vacunación, peluquería y cirugía.

Se eligió este tema porque permite trabajar de forma natural con las dos partes pedidas: un catálogo de productos con su proceso de compra, y una relación entre un cliente y un servicio que se concreta a través de una consulta.

2. Requerimientos funcionales

Se resolvieron los siguientes requerimientos funcionales:

- **RF1 – Consulta y compra de productos:** el cliente puede ver el catálogo en formato tabla y en formato tarjetas, abrir la ficha de un producto y completar un formulario de compra con nombre, dirección, teléfono, e-mail, medio de pago y listado de productos.
- **RF2 – Contacto y consultas:** el cliente puede enviar una consulta sobre un servicio completando un formulario. El sistema valida que los campos no estén vacíos y muestra en pantalla un mensaje de éxito o de error.

3. Mapa del sitio

El sitio está formado por seis páginas conectadas entre sí mediante un menú de navegación que se repite en todas ellas. La Tabla 1 resume la función de cada archivo.

Tabla 1

Archivo	Función
index.html	Portada con presentación de la veterinaria y acceso al catálogo.
listado_tabla.html	Listado de todos los productos en forma de tabla.
listado_box.html	Listado de todos los productos en forma de tarjetas (box).
producto.html	Ficha de un producto en particular.
comprar.html	Formulario de compra.
contacto.html	Formulario de contacto y consultas (con JavaScript).



Archivo	Función
style.css	Hoja de estilos compartida por todas las páginas.
app.js	Código JavaScript del módulo de contacto.
respuesta.json	Archivo local que simula la respuesta del servidor.

4. Diseño de las pantallas

Todas las páginas comparten la misma estructura: un encabezado con el nombre de la veterinaria y el menú, un área principal con el contenido de cada pantalla y un pie de página. Esto se logró con una única hoja de estilos (style.css), lo que mantiene un aspecto uniforme en todo el sitio, con una paleta basada en el color verde.

- **Portada (index.html):** muestra un mensaje de bienvenida y un botón que lleva al catálogo.
- **Listado en tabla (listado_tabla.html):** usa una tabla con código, producto, categoría, precio y un enlace a la ficha.
- **Listado en box (listado_box.html):** cada producto se presenta en una tarjeta con nombre, categoría, precio y un botón "Ver ficha". Las tarjetas se acomodan con flexbox.
- **Ficha de producto (producto.html):** detalla código, categoría, descripción, stock y precio, con botones para comprar o volver al listado.
- **Formulario de compra (comprar.html):** pide nombre del cliente, dirección, teléfono, e-mail, medio de pago (efectivo, tarjeta o transferencia) y permite elegir los productos a comprar.

Los productos de ejemplo utilizados en el catálogo se detallan en la Tabla 2.

Tabla 2

Productos del catálogo de ejemplo

Código	Producto	Categoría	Precio
1	Alimento balanceado perro adulto 15 kg	Alimentos	\$45.000
2	Alimento balanceado gato 7,5 kg	Alimentos	\$32.000
3	Collar antipulgas	Sanidad	\$8.500
4	Shampoo para mascotas	Higiene	\$6.200
5	Correa reforzada	Accesorios	\$7.000



5. Módulo de Contacto y Consultas con JavaScript

La página `contacto.html` contiene un formulario con nombre, e-mail, servicio (consulta general, vacunación, peluquería o cirugía) y el texto de la consulta. Para cumplir con los requisitos se aplicó lo siguiente:

- **Desacoplamiento:** el HTML no tiene atributos `onclick` ni `onsubmit`. El evento se captura desde `app.js` con `addEventListener("submit", ...)`, y se usa `e.preventDefault()` para que la página no se recargue al enviar.
- **Validación:** se leen los valores de los campos y se aplica `.trim()` para quitar espacios. Si el nombre, el e-mail o la consulta quedan vacíos, se muestra un mensaje de error en el DOM y no se envía nada.
- **Asincronía:** el envío se simula con la API `fetch()` usando `async/await` sobre el archivo local `respuesta.json`, que devuelve un mensaje de confirmación.
- **Mensaje al usuario:** si la respuesta es correcta se muestra un mensaje de éxito y se limpia el formulario; si `fetch` falla, un bloque `try/catch` muestra el error.

Como `fetch` necesita un servidor para leer archivos locales, la página debe abrirse con un servidor local (por ejemplo la extensión Live Server de Visual Studio Code) o desde GitHub Pages, y no con doble clic sobre el archivo.

6. Fundamentación del modelo EORM

En el modelo EORM, la relación entre dos entidades no es solo un vínculo que las une, sino un elemento con significado propio, capaz de guardar información. A esto se lo llama enlace enriquecido.

En Veterinaria Patitas, las entidades principales son el Cliente (con su nombre y e-mail) y el Servicio (consulta general, vacunación, peluquería, cirugía). Un cliente puede consultar por varios servicios, y un mismo servicio recibe consultas de muchos clientes. Esa relación es de muchos a muchos y, por sí sola, no dice nada más.

La Consulta es justamente el enlace enriquecido entre ambos: además de conectar a un cliente con un servicio, tiene datos propios, como el texto del mensaje, el momento en que se envía y su estado (recibida o respondida). Por eso se la modela como una relación con atributos propios y no como una simple conexión. El formulario de contacto lo refleja: el cliente completa sus datos, elige un servicio y escribe la consulta, que es la que une a ambos



CONCLUSIONES

Con este trabajo se logró armar un sitio web completo para Veterinaria Patitas, con el catálogo en dos formatos, la ficha de producto y el formulario de compra, sobre una estructura común y un diseño uniforme gracias a la hoja de estilos compartida.

El módulo de Contacto y Consultas permitió aplicar dos conceptos importantes de JavaScript: separar el comportamiento del HTML mediante `addEventListener` y trabajar con datos de forma asíncrona usando `fetch` con `async/await`. Mantener el código simple facilitó entender cada paso y poder explicarlo.

Por último, el modelo EORM ayudó a pensar la consulta como algo más que un formulario: es el enlace que une a un cliente con un servicio y que guarda información propia. Como mejora a futuro, la respuesta simulada con un archivo `.json` podría reemplazarse por un servidor real que almacene las consultas y las compras.



ANEXOS

Anexo 1. Estructura de archivos del repositorio

- index.html
- listado_tabla.html
- listado_box.html
- producto.html
- comprar.html
- contacto.html
- style.css
- app.js
- respuesta.json